PULSEROUTE

High-Performance Multimedia Transfer and Network Performance Laboratory

Engineering progress report - runtime validation incomplete

28 October 2026

Project owner: ISI0007

University, course and student identity are to be supplied by the author.

Evidence boundary: this is a progress report. Current Go runtime validation is blocked by Smart App Control. Compilation, frontend rendering and schema checks are distinct from full-stack verification.

Contents

1. Abstract

2. Introduction

3. Problem statement

4. Objectives

5. Literature and technology background

6. Functional requirements

7. Nonfunctional requirements

8. System architecture

9. TCP and UDP protocol design

10. Algorithms and formulas

11. Database design

12. Implementation

13. Testing methodology

14. Experimental results

15. Performance analysis

16. Security considerations

17. Windows execution-policy diagnosis

18. Limitations

19. Future improvements

20. Conclusion

21. References

22. Appendices

1. Abstract

PulseRoute is a Windows-focused Computer Networks laboratory built around real Go TCP and UDP sockets, a React desktop interface, and SQLite persistence. The implementation includes bounded framing, concurrent chat, streamed file uploads, SHA-256 verification, echo experiments and LAN discovery. This report describes the implementation and the evidence actually obtained. Earlier loopback core tests passed, and the current frontend, schema checks and compilation checks are successful. Smart App Control blocks current Go executable launches. Consequently, current full-stack runtime behavior, two-device performance and final demonstration readiness remain unverified. The project title describes an engineering target; it is not a measured claim of superior throughput.

2. Introduction

Computer networking concepts become easier to evaluate when an application exposes the relationship between transport guarantees and application responsibilities. TCP supplies an ordered byte stream but does not define message boundaries, file naming, persistence or user-visible completion. UDP exposes datagrams but does not inherently guarantee delivery or order. PulseRoute makes these distinctions explicit through communication and measurement workflows. The project separates transport code from the desktop service and database so that command-line experiments and a graphical demonstration share the same Go implementation.

3. Problem statement

A successful file copy alone does not explain framing, partial writes, integrity, cancellation, acknowledgment ambiguity or network measurement. A polished interface alone does not prove that real sockets are used. The project must combine observable networking behavior with reproducible evidence while avoiding claims unsupported by the environment. During implementation, storage exhaustion and Windows executable policy enforcement also became practical constraints. The engineering problem therefore includes preserving correctness and evidence when some validation routes are unavailable.

4. Objectives

The core objectives are multi-client text communication, binary-safe multimedia file transfer, direct LAN transfers, TCP/UDP experimentation, live progress and durable local records. Additional objectives are an accessible desktop interface, explicit error states, bounded resource usage, testable modules and academically defensible documentation. Video-file transfer is included; real-time video calling is not. Internet-wide peer-to-peer connectivity, NAT traversal, encrypted sessions and application implementations of TCP congestion-control algorithms are not part of the verified scope.

5. Literature and technology background

RFC 9293 defines TCP's transport behavior, while RFC 768 defines UDP. Go's net package exposes listeners, connections, deadlines and socket-oriented I/O. SHA-256 is specified in NIST FIPS 180-4 and is used here to compare complete file content. SQLite provides transactions and local persistence; WAL is used only on the local host. Wails connects a Go service to WebView2 on Windows. React, TypeScript and Tailwind implement the interface. These sources justify design choices, but documentation about a technology is not evidence that this application's use of it passed a test.

6. Functional requirements

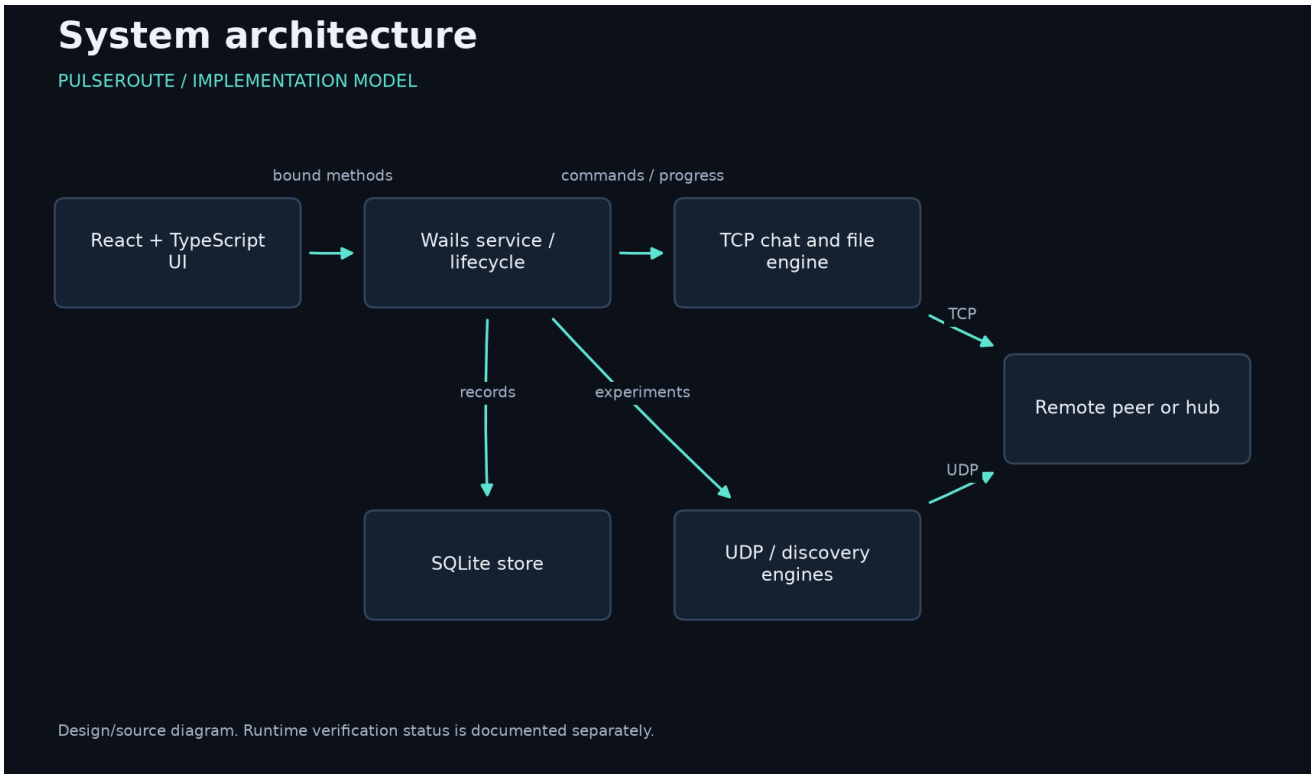
Operators must start and stop listeners, connect named clients, send messages, select and queue files, cancel a transfer, inspect integrity status, discover LAN peers, run experiments and inspect history. A saved configuration should survive restart. The current source implements these workflows, but desktop binding behavior and persistence through the Go driver still require approved runtime testing. The hub routes chat messages and can receive file uploads; forwarding a stored file from the hub to another client is not implemented. This remaining gap must be resolved or explicitly accepted before final submission.

7. Nonfunctional requirements

Frames are bounded to 128 KiB including their type byte, file chunks to 64 KiB, declared files to 8 GiB, and simultaneous TCP handlers to 64. Handshake, write, file-operation and heartbeat deadlines limit stalled work. The interface limits queue size, history snapshots, inline previews and retained progress. SQL uses parameters. Runtime success must be supported by tests rather than by successful compilation alone. Accessibility and readability are addressed through contrast, focus outlines, semantic controls and dark/light styling; a complete accessibility audit remains outstanding.

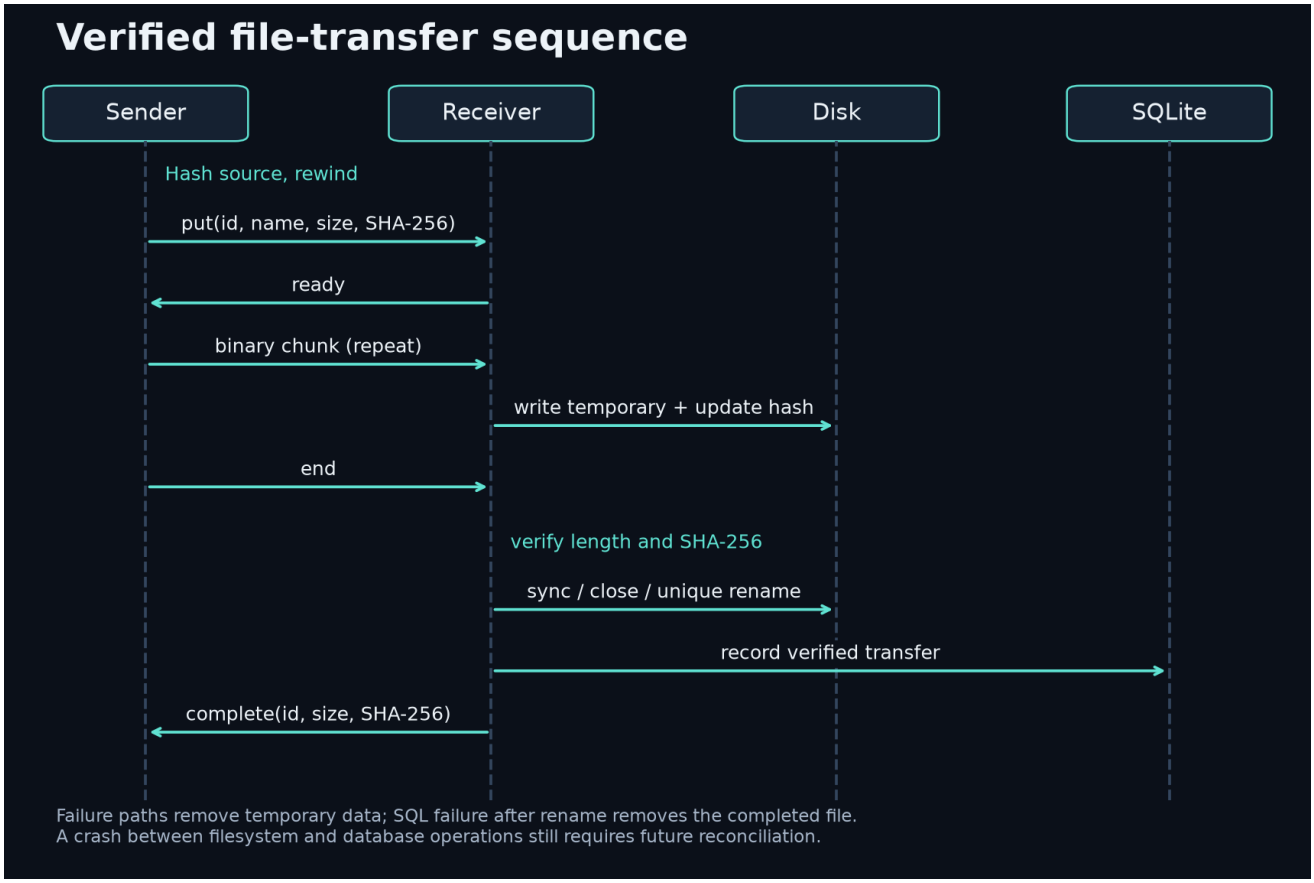
8. System architecture

The frontend calls exported Wails methods. The service coordinates lifecycle, connection state, transfer cancellation, experiments and persistence. TCP chat and file transfers use separate connections so a large upload does not occupy the chat stream. UDP probes and discovery use separate datagram operations. SQLite stores durable activity while progress remains transient. A node can host a listener and connect as a client. A dedicated chat hub is a listener shared by multiple clients. The architecture and deployment diagrams identify these boundaries without implying cloud services or internet NAT traversal.



9. TCP and UDP protocol design

The TCP envelope consists of a four-byte big-endian length, a one-byte frame type and a bounded payload. Control frames contain versioned JSON; file chunks are binary. Chat begins with hello/welcome and supports message, acknowledgment and heartbeat operations. File transfer uses put, ready, chunk frames, end and complete. The receiver checks declared size and SHA-256 before reporting verification. UDP echo probes include a magic value and sequence number. Discovery includes a nonce and derives the advertised address from the UDP source. Discovery results are hints on a trusted LAN, not authenticated identities.

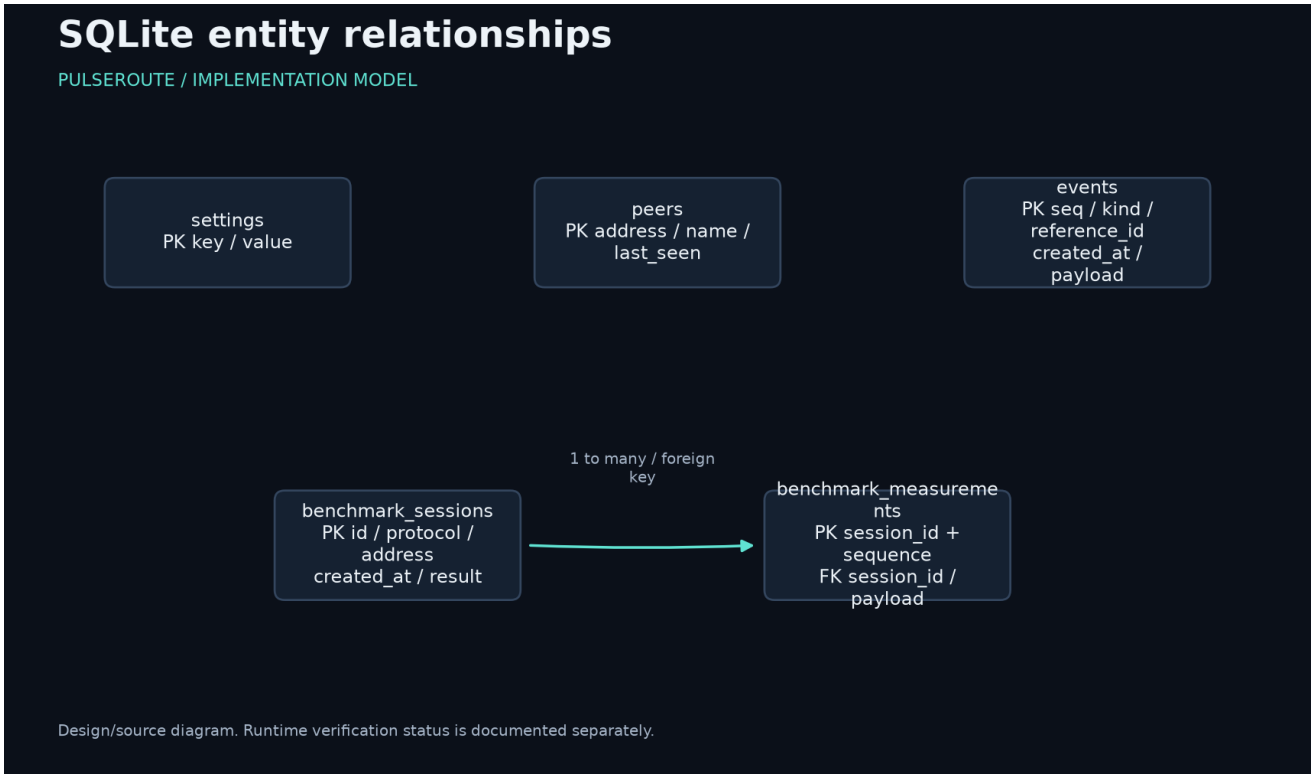


10. Algorithms and formulas

Framing is linear in frame size and bounded in allocation. File hashing and transmission are linear in file size with a fixed chunk buffer; the sender performs a preliminary hash pass and a second transfer pass. RTT is measured from local send to receive. Loss is the fraction of sent probes without a unique observed echo. Duplicate and out-of-order observations are derived from sequence numbers. RTT variation uses an exponential smoothing factor of 1/16. Echo goodput counts returned payload bytes over elapsed experiment time. Detailed equations, complexity and interpretation limits are supplied in docs/ALGORITHMS.md.

11. Database design

Schema version 1 contains five tables: settings, peers, events, benchmark_sessions and benchmark_measurements. The measurement table has a foreign key to its session. Message and transfer metadata are encoded in event payloads; the schema does not pretend these are separately normalized media tables. Indexes support event chronology and reference lookup. Startup restores validated node settings, known peers and recent results. Benchmark sessions and their raw observations are inserted transactionally. File rename and SQL insertion are separate resource operations, so a process crash can still leave an orphaned file; crash reconciliation is not implemented.



12. Implementation

The wire package reads complete frames and handles short writes. The server tracks connections, limits active handlers and closes sockets before waiting during shutdown. Chat writes are serialized per connection. The client sends periodic heartbeats and now bounds silent reads; handshake cancellation closes the socket. File reception writes to a random temporary path and renames only after verification. Desktop shutdown cancels service work and rejects new tracked operations before database close. Configuration is validated before persistence, and media preview reads are limited even if a file grows after its initial stat. These recent changes compile but still need runtime regression tests.

13. Testing methodology

Validation is separated into unit tests, loopback integration tests, SQL schema checks, frontend-only browser checks, static analysis, compilation, race checks and two-device experiments. The early test suite exercised framing bounds, short writes, eight clients, reconnect, empty files, chunk boundaries, duplicate names, corruption, interruption and cancellation. Current regression tests add settings/history restoration and progress retention. Python SQLite checks execute the actual SQL schema but do not validate the Go driver. Browser checks load the built frontend without a mock backend; they verify navigation and rendering, not native desktop communication.

14. Experimental results

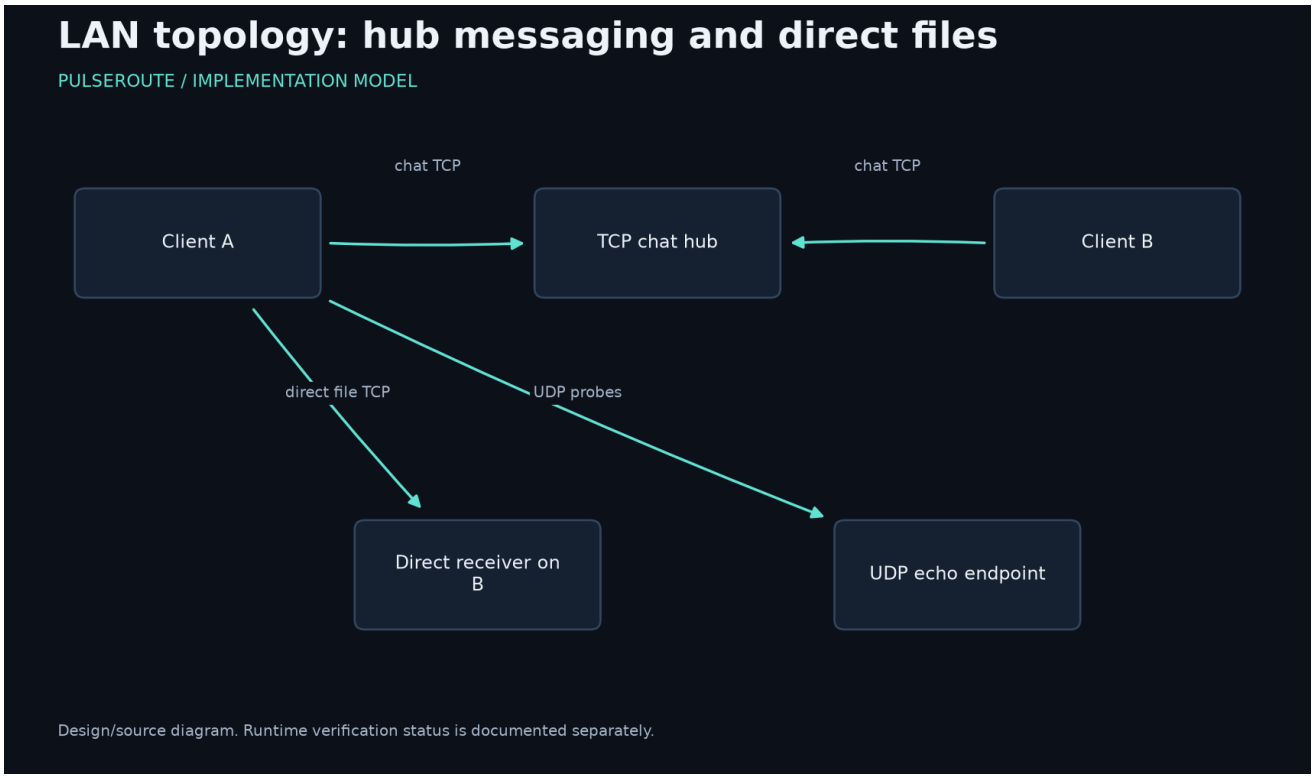
Earlier successful test assertions recorded eight simultaneous chat clients, verified binary transfers of 0, 1, 65,537 and 16,777,216 bytes (two copies per size), and 20 successful loopback echoes for each of TCP and UDP. These are correctness observations, not benchmark measurements. Current TypeScript/Vite builds, Go static analysis, Windows compilation, six-screen browser navigation and SQL schema checks pass. Current Go runtime execution is BLOCKED by Smart App Control. No successful raw RTT/throughput benchmark run was exported, so no performance chart or numerical speed comparison is presented.

15. Performance analysis

The architecture uses streaming to bound memory independently of file size, but that design property is not a measured optimization result. Frame allocations, synchronous database writes, hashing passes, disk throughput and serialized per-peer sends may affect performance. Sequential echo goodput is sensitive to round-trip delay and timeouts and cannot establish saturated link capacity. Loopback tests also omit physical links, switches, wireless contention and firewall behavior. A valid final analysis requires repeated trials with recorded topology, hardware, parameters and raw samples. No before/after speedup is claimed.

16. Security considerations

The default listener addresses are loopback. LAN use must be explicit and limited to trusted networks because authentication and encryption are not implemented. Remote filenames cannot select arbitrary directories; generated prefixes reduce collisions and prevent reserved-name conflicts. Temporary files are removed on ordinary failure. SHA-256 protects against accidental corruption but does not prevent a malicious sender from choosing both content and digest. Resource bounds reduce individual request impact, but aggregate disk quotas and rate limits are not implemented. Code signing and Windows policy approval are distinct from network protocol security.



17. Windows execution-policy diagnosis

Read-only event-log inspection obtained Code Integrity event 3077 for the CLI (record 14737) and a Go store test (record 14722). Both identify VerifiedAndReputableDesktop with policy GUID {0283ac0f-fff1-49ae-ada1-8a933130cad6}. The registry value is 1, indicating Smart App Control enforcement. This confirms the earlier probable diagnosis. No security controls were changed and blocked binaries were not retried unnecessarily. SAC has no per-app exception; the supported developer route requires valid code signing. Enterprise WDAC or AppLocker rule approval would apply only if those policies were separately identified.

18. Limitations

The project is not ready to be represented as a completed final submission. Current native workflows, Go persistence, recent shutdown/reconnect changes, two-device LAN behavior, race detection, maximum-size files and soak tests remain unverified. Server-mediated file forwarding/download, transfer resume, crash reconciliation, TLS, peer authentication and NAT traversal are absent. Inline preview is limited to supported formats and 16 MiB. Known peers may be offline. GitHub repository access still returns 404, so no remote commit is claimed. License selection remains pending the owner's explicit choice.

19. Future improvements

The next priority is an approved runtime environment followed by the full regression suite and real two-device experiments. Correctness failures must be fixed before optional features. Functional gaps include server-mediated file distribution, durable recovery and stronger peer identity. Further improvements may include a normalized file catalog, storage quotas, robust benchmark export/reload, accessibility testing and measured profiling. QUIC, encrypted channels, audio streaming and real-time video are optional later work, not substitutes for completing the existing core.

20. Conclusion

PulseRoute now has substantive networking source, a compiled Windows desktop interface, a versioned local database schema and a documented evidence trail. Early core tests demonstrated real loopback communication and file identity; later permitted checks verified frontend behavior, SQL schema properties and buildability. The Windows policy block is attributed to Smart App Control using actual event records. The remaining work is explicit. The defensible conclusion is progress toward a functional laboratory, not a claim that all course requirements or performance targets have already been met.

21. References

[1] IETF, RFC 9293, Transmission Control Protocol: https://www.rfc-editor.org/rfc/rfc9293

[2] IETF, RFC 768, User Datagram Protocol: https://www.rfc-editor.org/rfc/rfc768

[3] Go net package: https://pkg.go.dev/net

[4] NIST FIPS 180-4: https://csrc.nist.gov/pubs/fips/180-4/upd1/final

[5] SQLite transactions: https://www.sqlite.org/lang_transaction.html

[6] SQLite WAL: https://www.sqlite.org/wal.html

[7] Wails installation: https://wails.io/docs/gettingstarted/installation/

[8] Go race detector: https://go.dev/doc/articles/race_detector

[9] Microsoft SAC testing and events: https://learn.microsoft.com/en-us/windows/apps/develop/smart-app-control/test-your-app-with-smart-app-control

[10] Microsoft SAC FAQ: https://support.microsoft.com/en-us/windows/security/threat-malware-protection/smart-app-control-frequently-asked-questions

22. Appendices

The repository contains editable Mermaid diagrams, rendered diagram PNG/SVG files, frontend-only screenshots, schema/UI evidence JSON, historical loopback assertions, Windows event evidence, test source and reproducible scripts. docs/DEMO.md provides a live-demonstration sequence and an honest blocked-execution fallback. docs/PROGRESS.md is the authority for feature status. The final presentation uses actual screenshots and explicit status labels. No synthetic data is substituted for missing measurements.

Frontend evidence

Production frontend in Chrome; no mocked data; native Go bridge unavailable.

